

Unit - 4

Number Theory Problems

CRT (Chinese Remainder Theorem)

$$x \equiv 3 \pmod{5}$$

$$x \equiv 1 \pmod{7}$$

$$x \equiv 6 \pmod{8}$$

CRT only works if the modulos are relatively prime
 exp: 5 & 7, 7 & 8, 5 & 8 are coprime.
 H.C.F. is 1.

$$x \equiv b_1 \pmod{n_1}$$

$$x \equiv b_2 \pmod{n_2}$$

$$x \equiv b_3 \pmod{n_3}$$

b_i	N_i	x_i	$b_i N_i x_i$
b_1	$N_1 = \frac{N}{n_1} = n_2 n_3$	x_1	$b_1 N_1 x_1$
b_2	$N_2 = n_1 n_3$	x_2	$b_2 N_2 x_2$
b_3	$N_3 = n_1 n_2$	x_3	$b_3 N_3 x_3$

remainder $\nearrow$

$N = n_1 n_2 n_3$

$x = \sum_{i=1}^3 b_i N_i x_i \pmod{N}$

$N_i = \frac{N}{n_i}$

Inverse of N_i

b_i	$N_i = \frac{N}{a_i}$	a_i	$b_i N_i x_i$
3	$7 \times 8 = 56$	1	168
1	$5 \times 8 = 40$	3	120
6	$5 \times 7 = 35$	3	630

$$N = 5 \times 7 \times 8 = 280$$

$$56x_1 \equiv 1 \pmod{5}$$

$$x_1 \equiv 1 \pmod{5}$$

1 mod 5

$$\begin{array}{r} 0 \\ 5 \overline{) 1} \\ -0 \\ \hline 1 \end{array}$$

$$40x_2 \equiv 1 \pmod{7}$$

$$5x_2 \equiv 1 \pmod{7}$$

try 1, 2, (3)

$$15 \equiv 1 \pmod{7}$$

$$x_2 \equiv 3 \pmod{7}$$

$$35x_3 \equiv 1 \pmod{8}$$

$$3x_3 \equiv 1 \pmod{8}$$

1, 2 (3)

$$x_3 \equiv 3 \pmod{8}$$

when u divide 168 by 5, remainder is 3

when u divide 120 by 7, remainder is 1

when u divide 630 by 8, remainder is 6

Try out example:-

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{4}$$

$$x \equiv 1 \pmod{5}$$

$$x = 168 + 120 + 630 = 918$$

$$x \equiv 918 \pmod{280}$$

$$x \equiv 78 \pmod{280}$$

$$\begin{array}{r} 3 \\ 280 \overline{) 918} \\ \underline{840} \\ 78 \end{array}$$

$$78 \equiv 2 \pmod{5}$$

$$\equiv 1 \pmod{7}$$

$$\equiv 6 \pmod{8}$$

GCD Algorithms

Euclid's Algorithm:-

Euclid(a, b)

$$\boxed{a \geq b \geq 0}$$

1. if $b = 0$

2. return a.

3. else return (b, a mod b).

$$\textcircled{1} \quad \text{gcd}(24, 6)$$

$$= \text{gcd}(6, 0)$$

$$= 6$$

$$\textcircled{2} \quad \text{gcd}(18, 4)$$

$$= \text{gcd}(4, 2)$$

$$= \text{gcd}(2, 0)$$

$$= 2$$

$$\textcircled{3} \quad \text{gcd}(27, 1)$$

$$= \text{gcd}(1, 0)$$

$$= 1$$

Modular Arithmetic

The concepts of modular arithmetic are used in RSA algorithm.

① $x \equiv y \pmod{n}$ (If both x & y leave same remainder when divided by n)
eg $36 \equiv 24 \pmod{12}$

② $x \equiv y \pmod{n}$ (If n divide $(x-y)$)
eg $20 \equiv 3 \pmod{17}$ (17 divides $(20-3)=17$)

③ If $x \equiv y \pmod{n}$ & $a \equiv b \pmod{n}$, then
 $(x+a) \equiv (y+b) \pmod{n}$

eg $17 \equiv 4 \pmod{13}$ & $42 \equiv 3 \pmod{13}$
 $59 \equiv 7 \pmod{13}$

④ If $x \equiv y \pmod{n}$ & $a \equiv b \pmod{n}$ then
 $(x-a) \equiv (y-b) \pmod{n}$

eg $42 \equiv 3 \pmod{13}$ & $28 \equiv 2 \pmod{13}$
 $14 \equiv 1 \pmod{13}$

Lower Bound Theory

Lower Bound Theory is based on the concept of minimum time that is required to execute an algorithm.

- Lower Bound Theory is used to calculate minimum number of comparisons required to execute an algorithm.

According to Lower Bound Theory, for a lower bound $L(n)$ of an algorithm, it is not possible to have any algorithm (for a common problem) whose time complexity is less than $L(n)$ for random i/p.

Once lower bound is calculated, then we can compare it with the actual complexity of the algorithm & if their order are same, then we can declare our algorithm as optimal.

$$P \begin{cases} A_1 \\ A_2 \\ A_3 \end{cases}$$

Solve problem P by Algorithm A_1, A_2, A_3 and find min. time required & declare it as $L(n)$.

Now, that algorithm out of A_1, A_2, A_3 will be optimal, whose time is near to $L(n)$.

DATE: / /
PAGE NO.:
The techniques which are used by Lower Bound Theory for obtaining lower bounds are:-

- ① Comparison Tree
- ② Oracle & Adversary Argument
- ③ Techniques for the algebraic problem.

① Comparison Tree

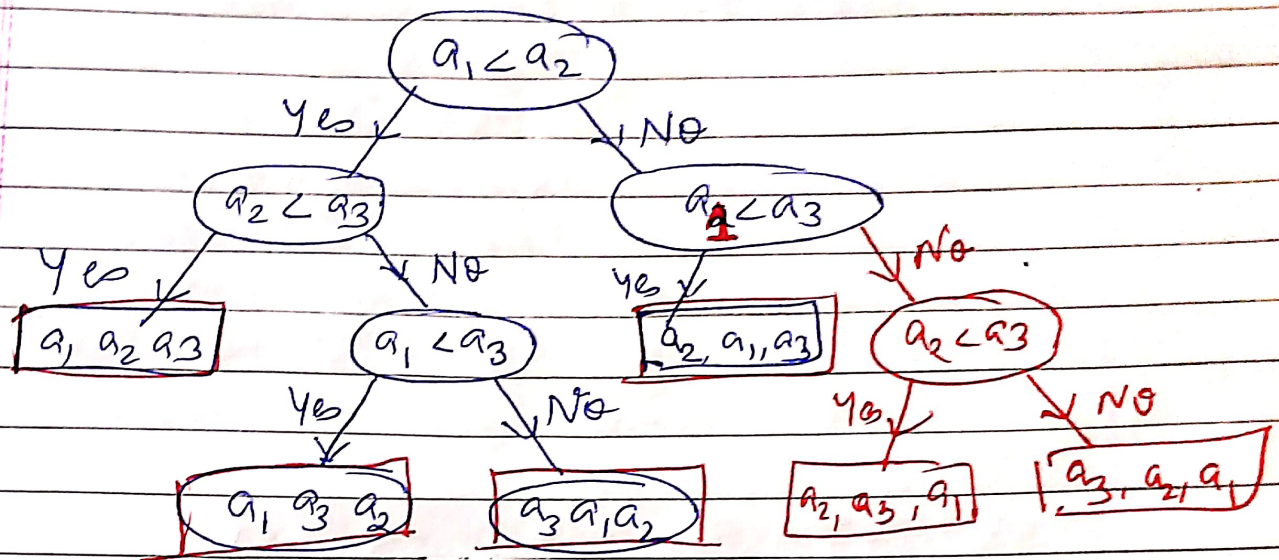
are the computational model useful for determining lower bounds for searching & sorting problems.

Comparison based algorithms are defined by a decision tree.

Decision Tree $\rightarrow$ A decision tree is a full binary tree that shows the comparisons b/w elements that are executed by an appropriate sorting algorithm operating on an i/p of a given size.

In a decision tree, there will be an array of length n , so total number leaves will be $n!$ (total number of comparisons).

Exp comparison of 3 elements a_1, a_2, a_3
 Left subtree will be true condition i.e. $a_i \leq a_j$
 Right subtree will be false condition i.e. $a_i > a_j$



6 cases

$$3! = 3 \times 2 \times 1 = 6$$

DBMS software oracle is also used for
 calculating Lower Bound.

Approximation Algorithms

Problems that don't have polynomial time solution (NP-hard), solving polynomial time sub-optimal solutions.

Vertex Cover Problem

Vertex Cover Problem for an undirected graph $G = (V, E)$ as a subset $V' \subseteq V$ such that if $(u, v) \in E$, then either $u \in V'$ or $v \in V'$ or both. It covers all the edges in E .

Vertex Cover Problem is NP-Complete.

Vertex Cover Problem is to find a vertex cover of minimum size.

Algorithm Approx Vertex Cover

Input to the algorithm is graph G .

Step 1 Initialize vertex cover C to be null
 $C \leftarrow \phi$.

Step 2 The set of edges in G is E .

Step 3 Repeat steps 4 to 6 till the set of edges E is empty

Step 4 choose an arbitrary edge (u, v) of E .

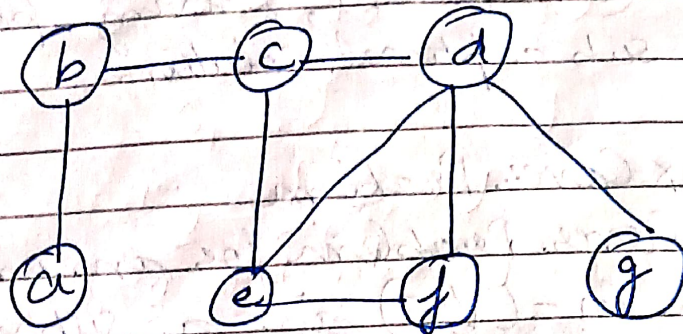
Step 5 Add the endpoints u, v to vertex cover C .

Step 6 Remove every edge incident on either u or v from the set of edges E .

Step 7 return C & Exit.

exp

For the graph, find approximate vertex cover.



Soln

Choose arbitrary edge (b,c) from graph

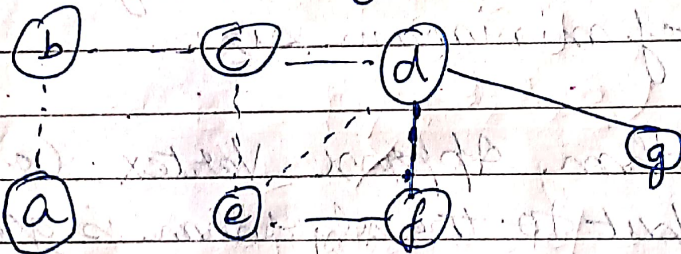
$$C = \{b, c\}$$

Remove all edges incident on either b or c

choose another edge (e,f)

$$C = \{b, c, e, f\}$$

Remove all edges incident on either e or f



Only edge left is (d,g)

$$C = \{b, c, e, f, d, g\}$$

No edges left in E

$$C = \{b, c, e, f, d, g\}$$

Size of approximate vertex cover is no more than size of optimal vertex cover.

$$\text{Optimal vertex cover} = \{b, d, e\}$$

Set Cover Problem

It is an optimization problem.

It is NP-hard.

Running time of set cover = $O(\log n)$.

For a finite set of elements X , there exists subsets of X , $S_1, S_2, \dots, S_m$. Each subset contains elements whose union gives X . Every element in X belongs to one of sets S_i , $i = 1$ to m .

Problem: Find min. no. of sets which contains all the elements of X .

Start with the set that covers maximum no. of elements. We follow greedy approach.

Algorithm Approximate - Set-Cover

Input: Sets S whose union is X .

Step 1 $C \leftarrow \phi$. set cover = empty.

Step 2 Set of elements covered from X is E
 $E \rightarrow \phi$.

Step 3 Repeat 4 & 5 while $E \neq X$,

Step 4 Select S_i that contains max. no. of uncovered elements. Add it to C

$$C \leftarrow C \cup S_i$$

Step 5 $E \leftarrow E \cup S_i$.

set of covered elements of S_i is added to E

Step 6 Return C & exit.

$$X = \{1, 2, 3, 4, 5\}$$

$$S = \{s_1, s_2, s_3\}$$

$$s_1 = \{4, 1, 3\}$$

$$s_2 = \{2, 5\}$$

$$s_3 = \{1, 4, 3, 2\}$$

$$C = \phi$$

$$E = \phi$$

choose s_3

$$C = s_3$$

$$E = \{1, 4, 3, 2\}$$

choose s_2

$$C = \{s_3, s_2\}$$

$$E = \{1, 4, 3, 2, 5\}$$

algorithm stops

$$\underline{\text{Exp: X}} = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8, e_9\}$$

$$\checkmark S_1 = \{e_1, e_2\}$$

$$S_2 = \{e_1, e_3, e_4\}$$

$$\checkmark S_3 = \{e_3, e_5, e_6\}$$

$$\checkmark S_4 = \{e_2, e_4, e_5, e_7, e_8\}$$

$$S_5 = \{e_6, e_7, e_9\}$$

$$\checkmark S_6 = \{e_8, e_9\}$$

$$C = \phi$$

$$E = \phi$$

choose S_4

$$C = \{S_4\}$$

$$E = \{e_2, e_4, e_5, e_7, e_8\}$$

choose S_3

$$C = \{S_3, S_4\}$$

$$E = \{e_2, e_3, e_4, e_5, e_6, e_7, e_8\}$$

$$\text{choose } S_1 \quad C = \{S_1, S_3, S_4\}$$

$$E = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8\}$$

$$\text{choose } S_6 \quad C = \{S_1, S_3, S_4, S_6\}$$

$$E = X.$$

$$A = \{1, 2, 3, 4, 5\} \text{ set}$$

$$B_1 = \{4, 1, 3\} \quad 5 \text{ cost}$$

$$B_2 = \{2, 5\} \quad 10 \text{ cost}$$

$$B_3 = \{1, 4, 3, 2\} \quad 3 \text{ cost}$$

Find that suitable subset that covers all elements of set with less cost,

$$B_1 \cup B_2 = \{4, 1, 3, 2, 5\} \quad 15 \text{ cost}$$

$$\boxed{B_2 \cup B_3 = \{1, 2, 4, 3, 5\} \quad 13 \text{ cost}}$$

$$B_1 \cup B_3 = \{1, 2, 3, 4\} \quad 8 \text{ cost but not covering all elements.}$$

$$B_1 \cup B_2 \cup B_3 = \{1, 2, 3, 4, 5\} \quad 18 \text{ cost}$$